KodeKloud

Cloud Native Observability › **Observability Fundamentals**

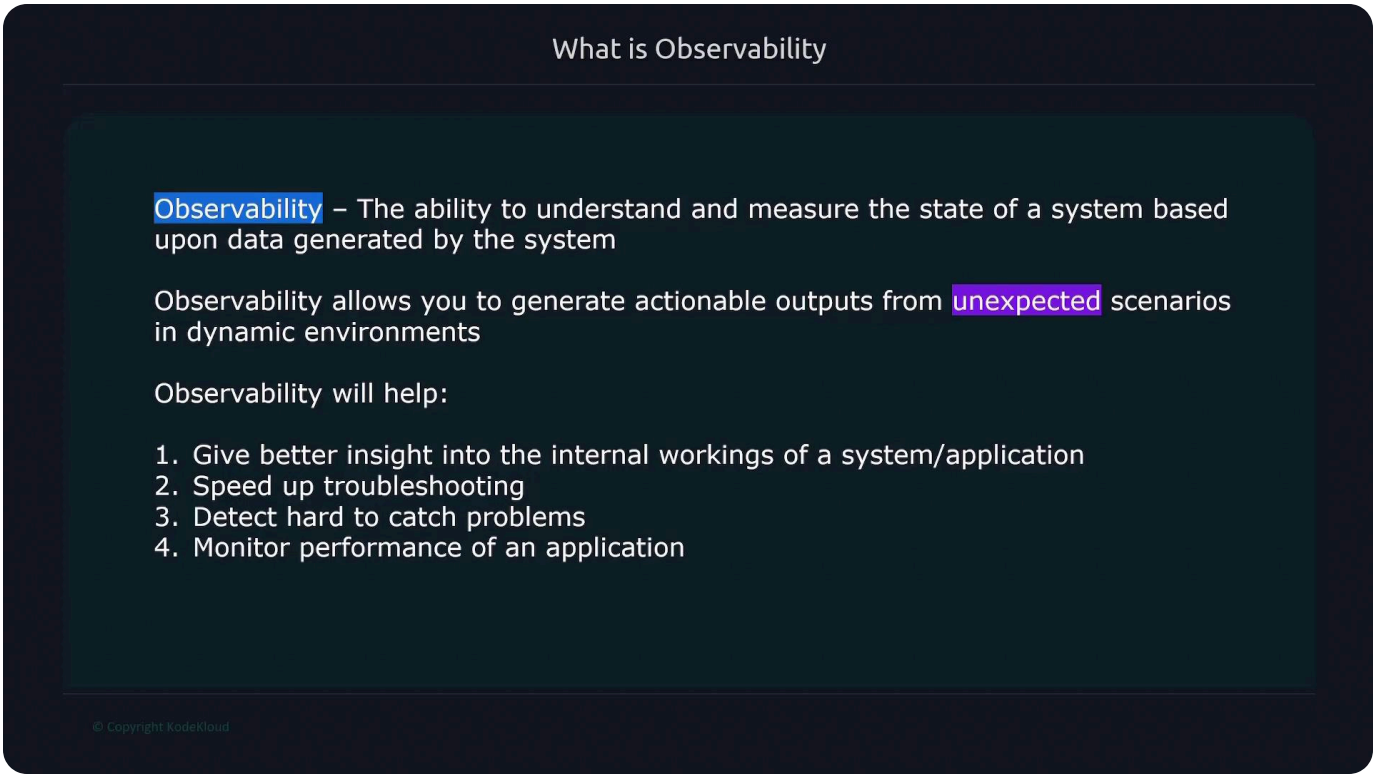
Cloud Native Observability

Observability Fundamentals

Copy page

This article explains observability, its importance in system monitoring, and the three pillars logging, tracing, and metrics for effective troubleshooting and performance insights.

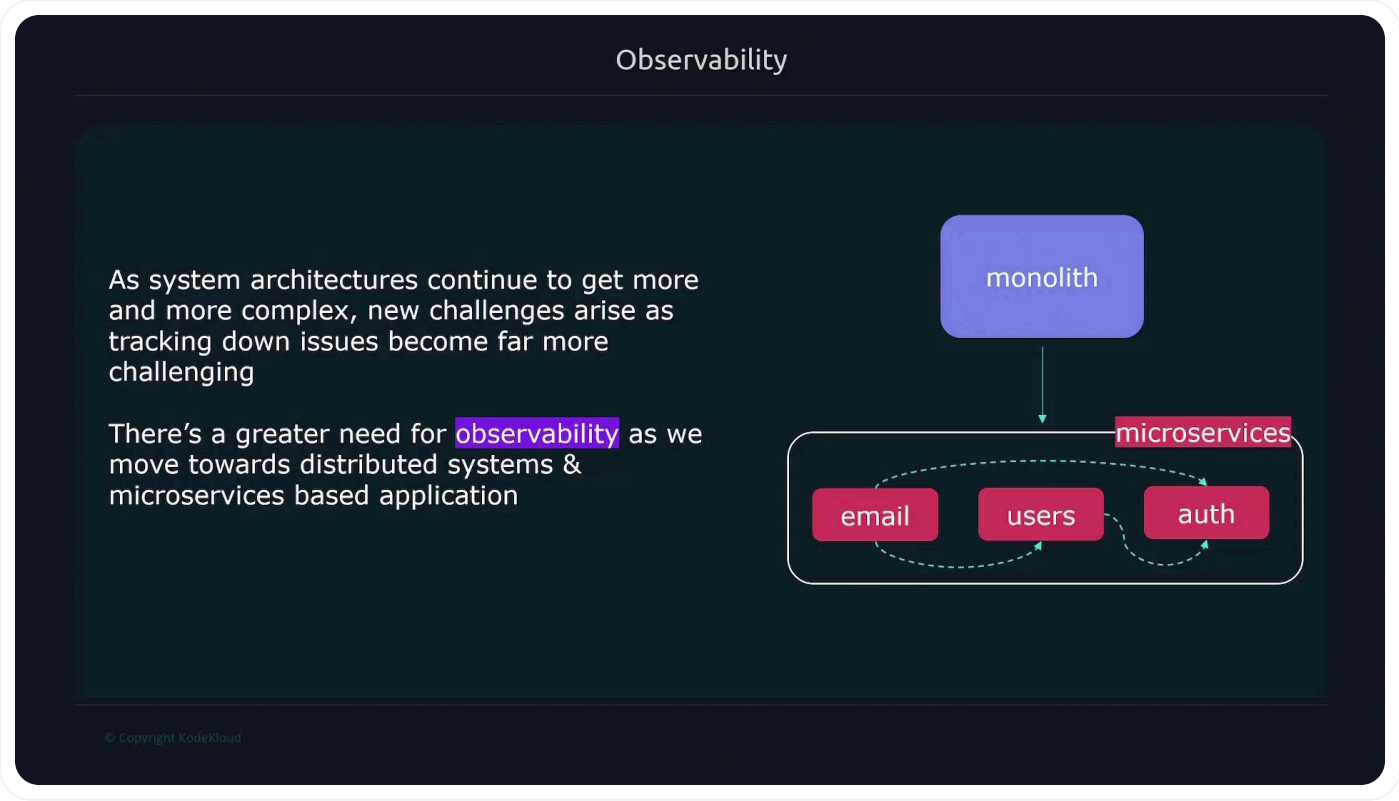
Observability is the capability to understand and measure a system's state based on the data it generates. By implementing observability within your applications and infrastructure, you gain deep insights into your system's internal workings. This, in turn, speeds up troubleshooting, detects elusive issues, monitors performance, and enhances cross-team collaboration.



Without observability, an application behaves like a black box—information enters and exits with little visibility into its internal processes. Observability lifts the veil, showing you how

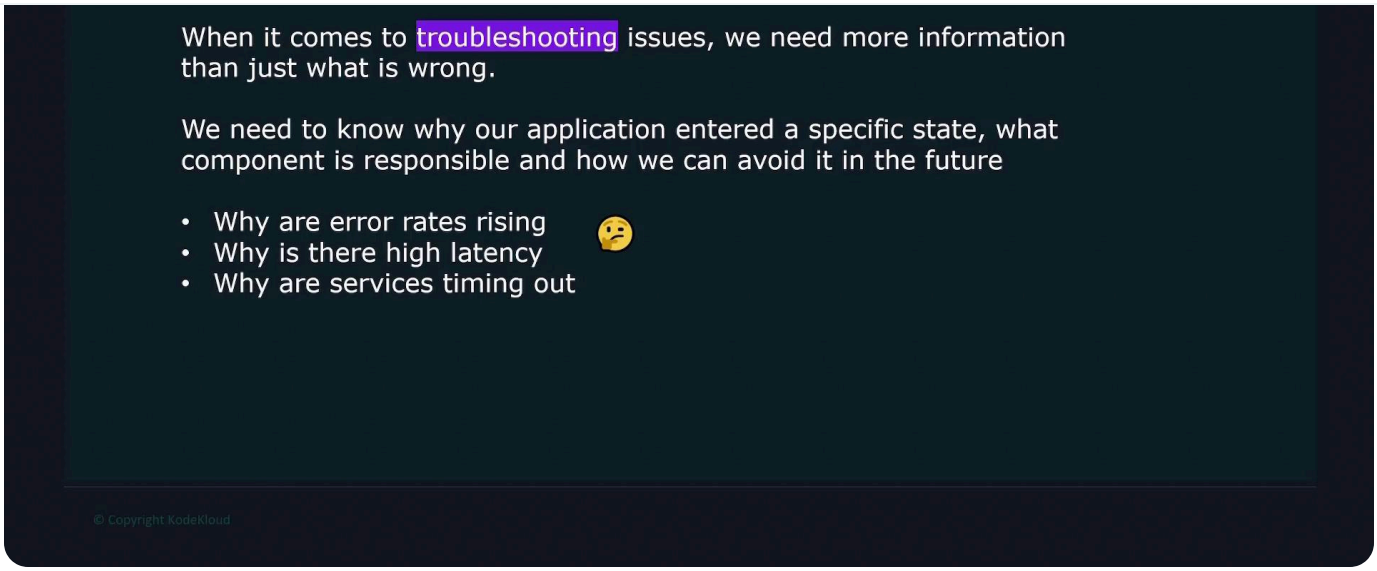
KodeKloud

infrastructure transforms from a single, monolithic system to a collection of independent, interacting services. This shift means you're no longer monitoring a unified entity but many small services working together. Such complexity can complicate troubleshooting, as isolating the failing service or component requires detailed insight.



Troubleshooting in these environments involves more than spotting symptoms. You need comprehensive data to understand why your application reached a specific state, which component is responsible, and how to mitigate future recurrences. For instance, you may seek answers to rising error rates, increased latency, or frequent service timeouts.

KodeKloud



Observability achieves this by leveraging three critical pillars: logging, tracing, and metrics.

Logging

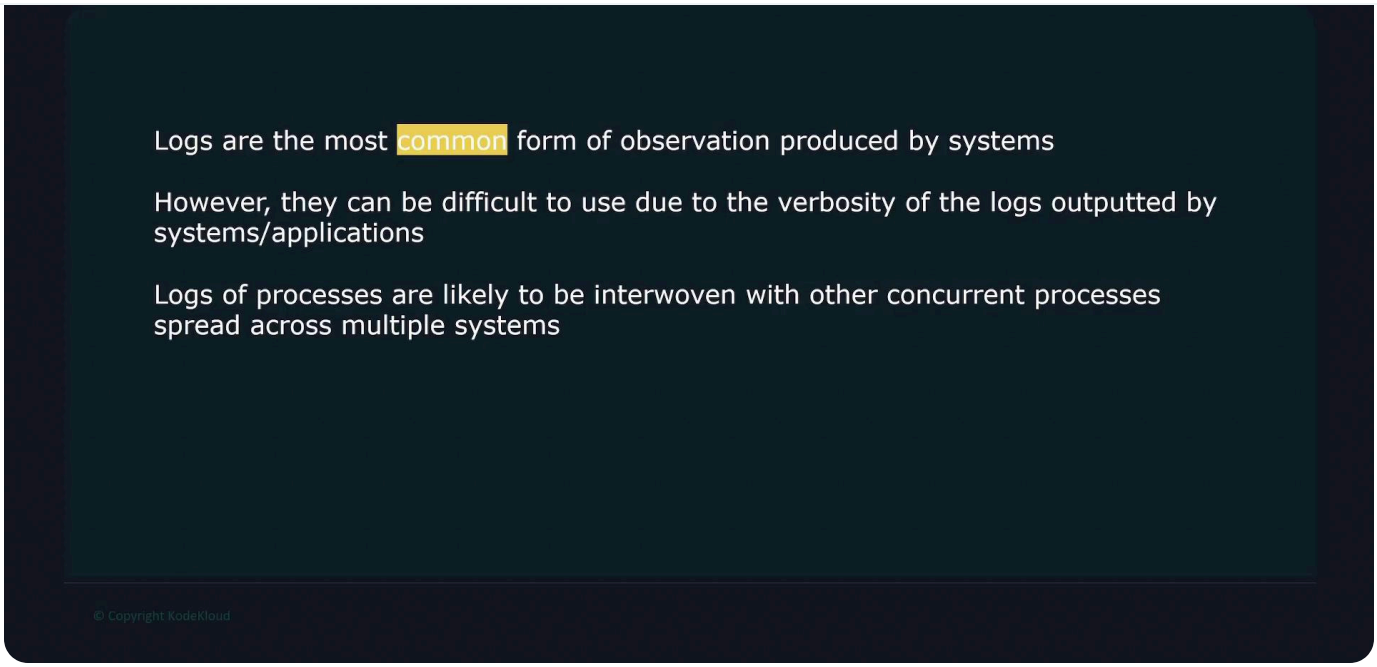
Logs are records of events that provide detailed information about system operations. Each log entry typically features a timestamp marking when the event occurred and a descriptive message. Logs are universally generated by operating systems, applications, and databases, serving as the first data point in your observability strategy.

For example, consider the following log entries:

```
Oct 26 19:35:00 ub1 kernel: [37510.942568] e1000: enp0s3 NIC Link
Oct 26 19:35:00 ub1 kernel: [37510.942697] e1000 0000:00:03.0 enp0s3: Reset adapt
Oct 26 19:35:03 ub1 kernel: [37513.054072] e1000: enp0s3 NIC Link is Up 1000 Mbps
```

While logs provide valuable contextual data, their high verbosity and the intertwining of processes across multiple systems can complicate the process of pinpointing issues during

KodeKloud



Tracing

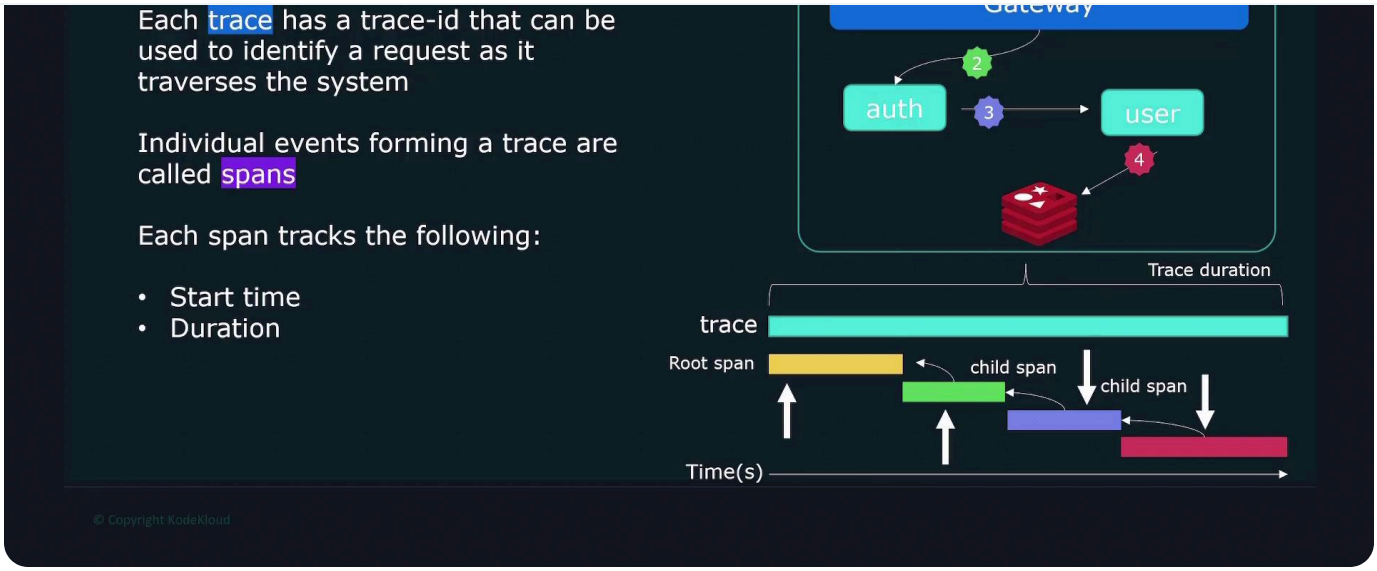
Tracing enables you to follow individual requests as they pass through various systems and services. Each request is assigned a unique trace ID, which allows you to visualize its journey across the entire application landscape. Within each trace, individual events—called spans—represent interactions at different interfaces or services. Each span records details such as start time, duration, and a parent ID that ties it back to the originating component.

For example, a request might generate:

- A span at the gateway.
- A span in the application layer.
- Additional spans when interacting with user services or databases.

These spans combine to form a complete trace of the request, providing a granular view of the interactions that occurred.

KodeKloud



Metrics

Metrics offer quantifiable measurements that reflect the state of a system. Unlike logs, which capture textual data, metrics deliver numerical data such as CPU load, the number of open files, HTTP response times, and error counts. These measurements can be aggregated over time and visualized, making it easier to detect trends and identify anomalies.

A typical metric entry might include:

A metric name that describes the measurement.

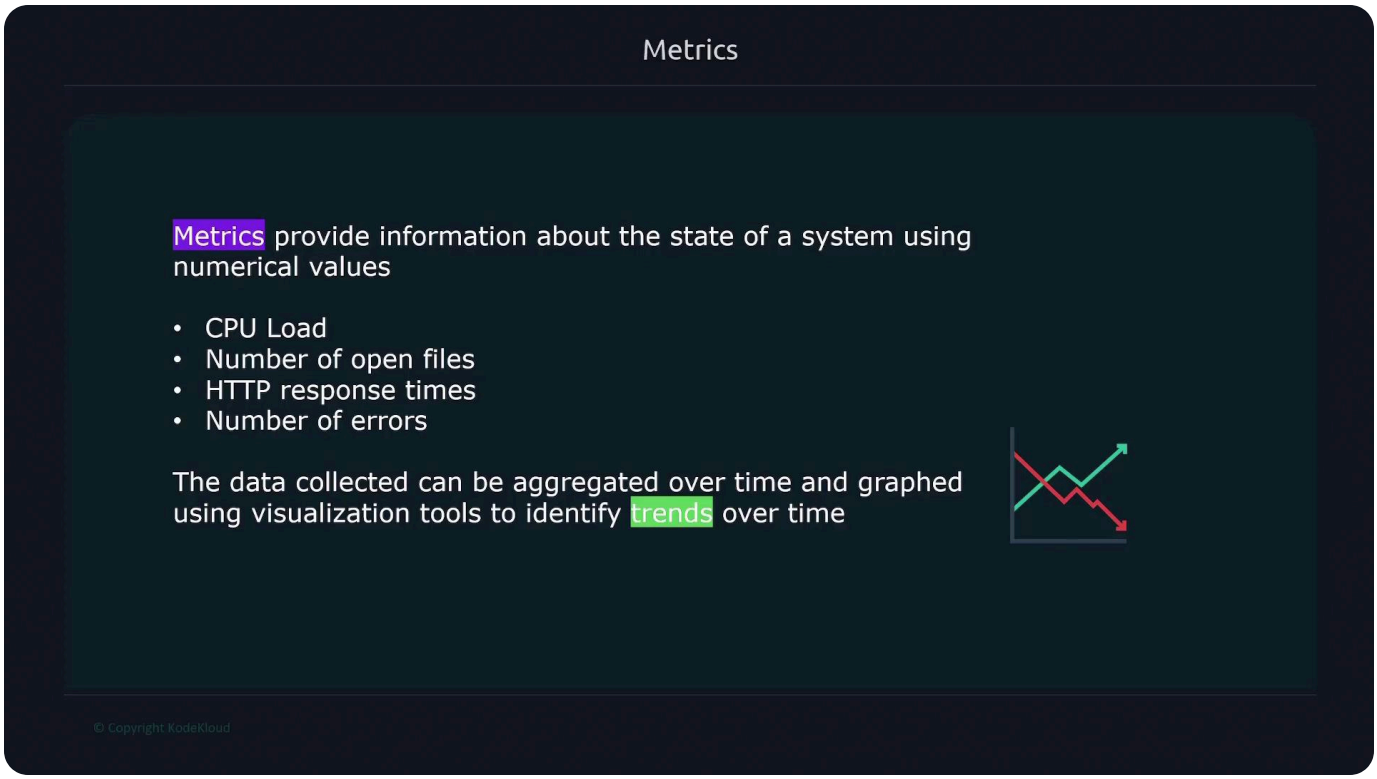
A value representing the current or recent reading.

A timestamp indicating when the metric was recorded.

Optional dimensions to provide additional context.

For example:

KodeKloud



> Observability is not limited to simply capturing data; its real power lies in correlating logs, traces, and metrics to gain a comprehensive view of your system's performance and health.

Observability with Prometheus

This article focuses on **Prometheus**, a leading monitoring solution designed for aggregating metrics. It's important to note, however, that Prometheus is specialized for handling metrics only—it does not capture logs or traces. To achieve a full observability solution, consider integrating additional tools for log management and distributed tracing.

By leveraging the three pillars of observability—logging, tracing, and metrics—you can develop a complete and robust system monitoring strategy. This holistic approach enables

KodeKloud

For more detailed insights and guides on observability, consider exploring additional resources:

- Prometheus Documentation
- Kubernetes Basics
- Docker Hub
- Terraform Registry

Watch Video

Learn more >

Was this page helpful? Yes No

Open Standards
< Previous

SLOSLASLI
Next >

KodeKloud